

RoboPet - HUD Polish

Polishes the top HUD: clears the OS status-bar overlap, moves currencies to their own row so they no longer collide with the care bars, adds an icon per bar, and animates bar fills + color changes. Code-only. Hand directly to Cursor.

Cursor task (build this now)

Replace the current CareHud layout with the version below.

1. Clear the status bar. Wrap the HUD in Align(topCenter) + SafeArea + Padding, and use Column(mainAxisSize: MainAxisSize.min). Remove the old Positioned(top: 12, ...).
2. Separate currencies. Add a currency row above the care bars (scrap on the left, gems on the right) so they no longer overlap the bars. Pull values from your economy provider.
3. Add icons. Give each stat bar a Material icon (restaurant / favorite / bolt) - no assets needed.
4. Animate the bars. Use TweenAnimationBuilder (450ms, Curves.easeOut) so fills and colors transition smoothly instead of jumping.
5. Keep the warning. Keep the low-stat (< 20) warning line under the bars.

Acceptance test

- HUD sits below the OS status bar (no clock/notch overlap)
- Currencies are on their own row and never overlap the bars
- Each bar shows an icon
- Feed / Pet / Charge animates the bar smoothly (no instant jump)
- Bar color shifts green -> amber -> magenta as the value drops
- Warning line still appears when a stat is < 20

1) CareHud layout

```
class CareHud extends ConsumerWidget {
  const CareHud({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final s = ref.watch(careProvider);
    final eco = ref.watch(economyProvider); // adapt to your economy state
    final warn = _firstCritical(s);
    return Align(
      alignment: Alignment.topCenter,
      child: SafeArea(
        child: Padding(
          padding: const EdgeInsets.fromLTRB(12, 8, 12, 0),
          child: Column(mainAxisSize: MainAxisSize.min, children: [
            // Row 1: currencies
            Row(children: [
              _Currency(icon: Icons.settings, value: eco.scrap), // scrap
              const Spacer(),
              _Currency(icon: Icons.diamond, value: eco.premiumGems), // gems
            ])
          ])
        )
      )
    );
```

```

    ]),
    const SizedBox(height: 8),
    // Row 2: care bars
    Row(children: [
      _StatBar(icon: Icons.restaurant, value: s.hunger),
      const SizedBox(width: 8),
      _StatBar(icon: Icons.favorite, value: s.happiness),
      const SizedBox(width: 8),
      _StatBar(icon: Icons.bolt, value: s.energy),
    ]),
    if (warn != null)
      Padding(
        padding: const EdgeInsets.only(top: 6),
        child: Text(warn, style: const TextStyle(
          color: Color(0xFFFF3CAC), fontSize: 12,
          fontWeight: FontWeight.bold)),
      ),
    ]),
  ),
);
}

String? _firstCritical(CareState s) {
  if (s.energy < 20) return 'Low battery! Charge your pet.';
  if (s.hunger < 20) return 'Your pet is hungry!';
  if (s.happiness < 20) return 'Your pet feels lonely.';
  return null;
}
}

```

2) Animated stat bar

```

class _StatBar extends StatelessWidget {
  const _StatBar({required this.icon, required this.value});
  final IconData icon;
  final int value;

  static Color _colorFor(double v) {
    if (v < 20) return const Color(0xFFFF3CAC); // critical
    if (v < 50) return const Color(0xFFFFB627); // warning
    return const Color(0xFF7CFF6B); // good
  }

  @override
  Widget build(BuildContext context) {
    return Expanded(
      child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
        Row(children: [
          Icon(icon, size: 13, color: Colors.white70),
          const SizedBox(width: 4),
          Text('$value', style: const TextStyle(
            color: Colors.white, fontSize: 11, fontWeight: FontWeight.w600)),
        ]),
        const SizedBox(height: 3),
        // Animates from the previous value to the new one on every change
        TweenAnimationBuilder<double>(
          tween: Tween(begin: 0, end: value.toDouble()),
          duration: const Duration(milliseconds: 450),
          curve: Curves.easeOut,
          builder: (context, v, _) => ClipRRect(

```

```

        borderRadius: BorderRadius.circular(6),
        child: LinearProgressIndicator(
          value: v / 100.0,
          minHeight: 8,
          backgroundColor: const Color(0x33FFFFFF),
          valueColor: AlwaysStoppedAnimation<Color>(_colorFor(v)),
        ),
      ),
    ),
  ],
);
}
}

```

3) Currency chip

```

class _Currency extends StatelessWidget {
  const _Currency({
    required this.icon,
    required this.value,
    this.color = const Color(0xFF22E1FF),
  });
  final IconData icon;
  final int value;
  final Color color;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
      decoration: BoxDecoration(
        color: const Color(0xAA0B0E1A),
        borderRadius: BorderRadius.circular(14),
        border: Border.all(color: color.withOpacity(0.6)),
      ),
      child: Row(mainAxisSize: MainAxisSize.min, children: [
        Icon(icon, size: 14, color: color),
        const SizedBox(width: 4),
        Text('$value', style: const TextStyle(
          color: Colors.white, fontSize: 12)),
      ]),
    );
  }
}

```

4) Notes

- SafeArea is what fixes the clock/notch overlap from the screenshot - it insets the HUD below the OS status bar.
- Currencies now live on their own row, so they can never sit on top of the bars again.
- Material icons need no assets; swap to custom icons later if you want the neon look.
- TweenAnimationBuilder animates from the current displayed value to the new one automatically - on first build it fills from 0, and every later change eases smoothly.
- Adapt eco.scrap / eco.premiumGems to your real economy state field names.

Next after this: real dog animation frames (idle/walk/eat/happy) so Feed/Pet reactions are actually visible, then a day/night sleep cycle.